

Homework 2

(Due date: 9 November 2024 at 10am)

For Q1, you need to submit your answer as a PDF file.

Question 1 (25%) - Analyze the following program and answer the following sub-questions:

```
.data
    Values BYTE 0FFh, 017h, 06Ch, 0F8h, 027h, 031h

.code
main PROC
    MOV AX, 0FF74H
    MOV BX, 0FFh
    MOV CX, 01H
    MOV DX, 0FFFFh
    CALL function_1
LABEL1:    MOV AX, 03FFH
    XOR AX, 0110b
    MOV CX, 003H
    CALL function_1
LABEL2:    SUB DX,DX
    MOV ESI, offset Values
    MOV BH,[ESI+2]
    XOR BH, 074H
LABEL3:    CALL function_2
LABEL4:    exit
main ENDP

function_1 PROC
    MOV BL,AL
    MOV DL,AL
Again:     ROL DL, CL
    SHR BL,1
    INC CL
    JNC Again
    NEXT:  ret
function_1 ENDP

function_2 PROC
    MOV BL,AL
    MOV AL,-7
    CBW
    MOV DX,AX
    MOV AL,95
    CBW
    Add AX, DX
    ret
function_2 ENDP
END main
```

a. When the control reaches LA-BEL1 (just before executing “MOV AX, 03FFH”), what will be the values of the following registers:

Register, Memory location, or flag bit	Value of the register or memory location (in HEX)
AX	FF74
BX	000E
CX	0004
DX	FF1D

b. (4 points) When the control reaches LABEL2 (just before executing “SUB DX,DX”), what will be the values of the following registers:

Register, Memory location, or flag bit	Value of the register or memory location (in HEX)
AX	03F9
BX	007C
CX	0004
CF	1

c. (4 points) When the control reaches LABEL3 (just before executing “CALL function_2”), what will be the values of the following registers and flags:

Register, Memory location, or flag bit	Value of the register or memory location (in HEX)
BX	187C
CF	0
ZF	0
SF	0

d. (4 points) When the control reaches LABEL4 (just before executing “exit”), what will be the values of the following registers and flags:

Register, Memory location, or flag bit	Value of the register or memory location (in HEX)
AX	0058
BX	18F9
DX	FFF9
CF	1

For Q2-Q5, you need to submit the folder that includes the code of each question as a zip file. You will lose 30% of the grade if the code does not compile and run correctly.

Question 2 (30%): Write an assembly program that asks the user to enter 10 unsigned decimal numbers and store them in a DWORD array called **A**. The program should do the following:

1. The program should compute the sum of all even numbers and store it in a DWORD variable **SUM**. You need to print **SUM**. The program should also compute the product of all odd numbers and store it a DWORD variable **PROD**. You need to print **PROD**. (Note: you need to use the **MUL** and **ROR** instructions). Assume that the user enters at least one odd number and one even number.
3. The program should count the number of prime numbers entered by the user and store the count in the DWORD variable **COUNT**. You need to print **COUNT**. (Note: you need to use nested loops).

Below is a sample of the expected output:

```
Enter a decimal number: 5
Enter a decimal number: 12
Enter a decimal number: 5
Enter a decimal number: 3
Enter a decimal number: 4
Enter a decimal number: 77
Enter a decimal number: 8
Enter a decimal number: 7
Enter a decimal number: 12
Enter a decimal number: 13
The sum of even numbers is: +36
The product of odd numbers is: +525525
The the number of prime numbers is : +5
Press any key to continue...
```

Code: -

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a decimal number: ", 0
str2 byte "The sum of even numbers is: ", 0
str3 byte "The product of odd numbers is: ", 0
str4 byte "The number of prime numbers is: ", 0
A dword 10 DUP(?)
SUM dword 0
PROD dword 1
COUNT dword 0
```

.code

main PROC

```
mov edx, offset str1
```

```
mov esi, offset A
mov ecx, lengthof A
```

first:

```
call writestring
call readint
mov [esi], eax
add esi, 4
loop first
```

```
    mov esi, offset A
    mov ecx, lengthof A

second:
    mov eax, [esi]
    test eax, 1
    jz evenNum

    mul PROD
    mov PROD, eax
    jmp repeatLoop

evenNum:
    add SUM, eax

repeatLoop:
    add esi, 4
    loop second

    mov edx, offset str2
    call writestring
    mov eax, SUM
    call writeint
    call crlf

    mov edx, offset str3
    call writestring
    mov eax, PROD
    call writeint
    call crlf

    mov PROD, 1
    mov esi, offset A
    mov ecx, lengthof A

checkPrime:
    mov eax, [esi]

    test eax, 1
    jz continue

    mul PROD
    mov PROD, eax
    ror PROD, 1

continue:
    mov edx, eax
    cmp edx, 1
    jbe notPrime

    and edx, 1
    jz notPrime

    mov ecx, 3

sqrtLoop:
    mov eax, edx
    div ecx
    cmp edx, 0
    jz notPrime
    inc ecx
    cmp ecx, edx
    jge primeFound
    jmp sqrtLoop
```

```
notPrime:
    jmp continueLoop

primeFound:
    inc COUNT

continueLoop:
    add esi, 4
    loop checkPrime

    mov edx, offset str4
    call writestring
    mov eax, COUNT
    call writeint
    call crlf

    exit
main ENDP
END main
```

Output: -

```
Enter a decimal number: 5
Enter a decimal number: 12
Enter a decimal number: 5
Enter a decimal number: 3
Enter a decimal number: 4
Enter a decimal number: 77
Enter a decimal number: 8
Enter a decimal number: 7
Enter a decimal number: 12
Enter a decimal number: 13
The sum of even numbers is: +36
The product of odd numbers is: +525525
The number of prime numbers is: +5
```

Question 3 (15%): Write an assembly program that asks the user to enter 10 signed decimal numbers and store them in a DWORD array called **A**. The program should find and print the first element in the array that is multiple of 5 and 7 (if such element exists) along with its index. If the array elements do not include a multiple of 5 and 4, then the program should display a message stating that “No multiple of 5 and 7 is entered”.

Below is a sample of the expected output:

```
Enter a decimal number: -5
Enter a decimal number: 10
Enter a decimal number: 15
Enter a decimal number: 30
Enter a decimal number: -70
Enter a decimal number: 35
Enter a decimal number: 100
Enter a decimal number: 140
Enter a decimal number: 2
Enter a decimal number: 5
The first element that is multiple of 5 and 7 is -70
Press any key to continue..._
```

Code: -

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter a decimal number: ", 0
str2 byte "The first element that is multiple of 5 and 7 is ", 0
str3 byte " at index ", 0
str4 byte "No multiple of 5 and 7 is entered", 0
A dword 10 DUP(?)
i dword 0
```

.code

main PROC

```
mov edx, offset str1
```

```
mov esi, offset A
mov ecx, lengthof A
```

fillA:

```
call writestring
call readint
mov [esi], eax
add esi, 4
loop fillA
```

```
mov esi, offset A
mov ecx, lengthof A
```

findMult:

```
mov eax, [esi]
cdq
```

```
mov ebx, 5
idiv ebx
cmp edx, 0
jnz notDivisible
```

```
mov eax, [esi]
cdq
```

```
mov ebx, 7
idiv ebx
cmp edx, 0
jnz notDivisible
```

```
mov edx, offset str2
call writestring
mov eax, [esi]
call writeint
```

```
mov edx, offset str3
call writestring
mov eax, i
call writedec
call crlf
jmp endProgram
```

notDivisible:

```
inc i
add esi, 4
loop findMult
```

```
        mov edx, offset str4
        call writestring
        call crlf
```

```
endProgram:
```

```
        exit
main ENDP
END main
```

Output: -

```
Enter a decimal number: -5
Enter a decimal number: 10
Enter a decimal number: 15
Enter a decimal number: 30
Enter a decimal number: -70
Enter a decimal number: 35
Enter a decimal number: 100
Enter a decimal number: 140
Enter a decimal number: 2
Enter a decimal number: 5
The first element that is multiple of 5 and 7 is -70 at index 4
```

Question 4 (20%):

A. Complete the procedure below to swap (exchange) the values of [ESI] and [EDI] if the value of [ESI] is greater than [EDI].

```
;-----
SWAP PROC
;
; SWAP the values [ESI] and [EDI] if [ESI] > [EDI]
; Inputs:   ESI, EDI
; Returns:  nothing
; Requires: nothing
;-----
```

```
ret
SWAP ENDP
```

B. Define two DWORD arrays called **A** and **B**. You need to read the elements of arrays A and B (signed values). You need to use the SWAP procedure to implement using assembly the following C++ code:

```
for (i=0; i<10; i++)
    if (A[i] > B[i])
        swap(A[i], B[i]);
```

You need to print the elements of arrays A and B after running the above code.

Below is a sample of the expected output:

```
Enter the elements of array A
55
-2
-6
12
4
77
Enter the elements of array B
2
-1
9
-5
-8
1
The array A is:
+2
-2
-6
-5
-8
+1
The array B is:
+55
-1
+9
+12
+4
+77
```

Code: -

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
A dword 6 DUP(?)
B dword 6 DUP(?)
str1 byte "Enter the elements of array A", 0
str2 byte "Enter the elements of array B", 0
str3 byte "The array A is:", 0
str4 byte "The array B is:", 0
```

.code

main PROC

```
mov edx, offset str1
call writestring
call crlf
```

```
mov esi, offset A
mov ecx, lengthof A
```

fillA:

```
call readint
mov [esi], eax
add esi, 4
loop fillA
```

```
mov edx, offset str2
call writestring
call crlf
```

```
mov edi, offset B
mov ecx, lengthof B
```

fillB:

```
call readint
mov [edi], eax
add edi, 4
loop fillB
```

```
mov esi, offset A
```

```

    mov edi, offset B
    mov ecx, 6

swapLoop:
    call SWAP
    add esi, 4
    add edi, 4
    loop swapLoop

    mov edx, offset str3
    call writestring
    call crlf

    mov esi, offset A
    mov ecx, lengthof A

printA:
    mov eax, [esi]
    call writeint
    call crlf
    add esi, 4
    loop printA

    mov edx, offset str4
    call writestring
    call crlf

    mov edi, offset B
    mov ecx, lengthof B

printB:
    mov eax, [edi]
    call writeint
    call crlf
    add edi, 4
    loop printB

    exit
main ENDP

; -----
; SWAP PROC
;
;     SWAP the values [ESI] and [EDI] if [ESI] > [EDI]
;     Inputs: ESI, EDI
;     Returns: nothing
;     Requires: nothing
; -----

    mov eax, [esi]
    cmp eax, [edi]
    jle noSwap

    mov ebx, [esi]
    mov eax, [edi]
    mov [edi], ebx
    mov [esi], eax

noSwap:
    ret
SWAP ENDP
END main

```

Output: -

```
Enter the elements of array A
55
-2
-6
12
4
77
Enter the elements of array B
2
-1
9
-5
-8
1
The array A is:
+2
-2
-6
-5
-8
+1
The array B is:
+55
-1
+9
+12
+4
+77
```

Question 5 (10%): Write a program that asks the user to enter a signed decimal value and store it in register EBX. The program should analyze the binary bits of EBX and print the following:

- Compute and print the 2's complement of EBX in HEX format. You need to use the **XOR** instruction.
- Count and print the number of bits that have a value equal to 1. You need to use the **ROR** instruction.
- Set (set to 1) the bits of the second and third bytes of EBX and print the result, then print the value of EBX in HEX format. You need to use the **OR** instruction.

Below is a sample of the expected output:

```
Enter the value of EBX as a signed decimal number: 1015
The 2s complement of EBX is : FFFFC09
The number of EBX bits that have a value equal to 1 is : +9
The value of EBX after setting the second and third bytes is : 00FFFF7
Press any key to continue..._
```

Code: -

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 byte "Enter the value of EBX as a signed decimal number: ", 0
str2 byte "The 2s complement of EBX is: ", 0
str3 byte "The number of EBX bits that have a value equal to 1 is: ", 0
str4 byte "The value of EBX after setting the second and third bytes is: ", 0
```

.code

main PROC

```
mov edx, offset str1
call writestring
call readint
mov ebx, eax
```

```
mov edx, offset str2
call writestring
xor eax, 0FFFFFFFFh
inc eax
call writehex
call crlf
```

```
mov edx, offset str3
call writestring
```

```
mov ecx, 32
mov eax, 0
```

countBits:

```
ror ebx, 1
jc bitIsOne
jmp continueCount
```

bitIsOne:

```
inc eax
```

continueCount:

```
loop countBits
```

```
call writeint
call crlf
```

```
    mov edx, offset str4
    call writestring
    or ebx, 000FFFF00h
    mov eax, ebx
    call writehex

    call crlf

    exit
main ENDP
END main
```

Output: -

```
Enter the value of EBX as a signed decimal number: 1015
The 2s complement of EBX is: FFFFC09
The number of EBX bits that have a value equal to 1 is: +9
The value of EBX after setting the second and third bytes is: 00FFFFFF7
```